

Sistemas Distribuídos — Aula 27

Revisão Geral do 2º Bimestre e Orientação do Trabalho Final · Prof. Leandro Borges

Por que uma aula de revisão agora

Com a Aula 26 fechamos o bloco de sistemas operacionais distribuídos e, junto com ele, todo o conteúdo expositivo do 2º bimestre. Esta aula tem dois objetivos combinados: primeiro, organizar em um único painel tudo o que foi visto desde a computação em grid até os sistemas de arquivos distribuídos, destacando os conceitos que mais aparecem em avaliação; segundo — e igualmente importante — fazer um ponto de situação do trabalho final, já que a partir de agora o foco prático da disciplina passa a ser o desenvolvimento do último checkpoint do projeto evolutivo, com apresentação marcada para a Aula 31, em 15/12.

Mapa do 2º bimestre

O bimestre começou com computação em grid e cluster (Aulas 14-15): a ideia de agregar o poder computacional de várias máquinas, sejam elas de um mesmo cluster fisicamente próximo ou de um grid geograficamente distribuído, para viabilizar processamento de alto desempenho que uma única máquina não realizaria sozinha.

Em seguida vieram computação móvel e comunicação em tempo real (Aulas 16-17): como um sistema distribuído lida com nós que mudam de localização (roaming), como o roteamento se adapta a essa mobilidade e quais exigências adicionais de latência e continuidade aparecem em aplicações de comunicação on-line e em tempo real.

O núcleo teórico mais denso do bimestre veio com os algoritmos distribuídos avançados (Aulas 18-20): consenso distribuído — como um conjunto de processos concorda em um valor mesmo na presença de falhas —, snapshot distribuído — como capturar uma "fotografia" consistente do estado global de um sistema sem interromper sua execução — e replicação e consistência de dados — as diferentes estratégias para manter cópias de um mesmo dado sincronizadas, e os modelos de consistência que cada estratégia viabiliza.

Na sequência, tolerância a falhas (Aulas 21-23) tratou de como um sistema continua funcionando, ou falha de forma controlada, quando componentes individuais param de responder: os diferentes modelos de falha, as técnicas de detecção, as estratégias de replicação e redundância como mecanismo de proteção, e os procedimentos de recuperação depois que uma falha ocorre.

Por fim, sistemas operacionais distribuídos (Aulas 24-26) trouxe a camada de infraestrutura que sustenta tudo isso na prática: os conceitos e arquiteturas de um SO pensado para múltiplas máquinas, a gerência de processos e o escalonamento distribuído entre elas, e os sistemas de arquivos distribuídos que permitem que dados sejam acessados de forma transparente independentemente de onde estejam fisicamente armazenados.

CAP Theorem e Consenso Distribuído (Paxos e Raft)

O CAP Theorem é, provavelmente, o conceito mais cobrado de todo o bimestre: ele afirma que um sistema distribuído não consegue garantir simultaneamente Consistência (todos os nós veem os mesmos dados ao mesmo tempo), Disponibilidade (toda requisição recebe uma resposta) e Tolerância a Partição (o sistema continua funcionando mesmo que a comunicação entre alguns nós seja interrompida) — no máximo duas das três ao mesmo tempo. Como partições de rede são um fato da vida em qualquer sistema distribuído real, a tolerância a partição

normalmente não é negociável, o que reduz a decisão de projeto, na prática, a escolher entre priorizar consistência ou priorizar disponibilidade quando uma partição realmente acontece.

Paxos e Raft são os dois algoritmos de consenso mais citados na literatura e mais cobrados em prova. Ambos resolvem o mesmo problema — fazer um conjunto de réplicas concordar em um único valor mesmo com falhas de nós ou de comunicação — mas com filosofias diferentes. Paxos, formulado por Leslie Lamport, é o algoritmo clássico e matematicamente elegante, porém reconhecidamente difícil de entender e de implementar corretamente. Raft foi desenhado explicitamente para ser mais compreensível, mantendo as mesmas garantias de correção, através de uma decomposição clara em eleição de líder, replicação de log e segurança: um líder é eleito entre as réplicas, ele recebe as escrituras e replica cada entrada de log para os seguidores, e o sistema só considera uma entrada confirmada quando a maioria das réplicas a registrou.

Snapshot Distribuído e Modelos de Falha

O snapshot distribuído, no algoritmo de Chandy-Lamport, resolve um problema sutil: como obter uma imagem consistente do estado global de um sistema — o estado de cada processo mais as mensagens em trânsito em cada canal — sem parar a execução do sistema para tirar essa "foto". A solução usa marcadores (markers) especiais, propagados pelos canais de comunicação: quando um processo recebe o primeiro marcador em qualquer canal, ele registra seu próprio estado local e passa a gravar as mensagens que chegam pelos demais canais até receber o marcador neles também. O resultado é um snapshot consistente mesmo que os processos estejam em pontos diferentes de sua execução no momento da captura — útil, por exemplo, para detecção de terminação distribuída e para depuração de sistemas em produção.

Os modelos de falha classificam o que pode dar errado em um sistema distribuído, e cada modelo pede uma estratégia de tolerância diferente. A falha por parada (crash failure) é a mais simples: um processo para de responder e não volta mais. A falha por omissão ocorre quando mensagens são perdidas — no envio ou no recebimento — sem que o processo em si tenha parado. A falha de temporização (timing failure) acontece quando uma resposta chega, mas fora da janela de tempo esperada, o que é especialmente relevante em sistemas com restrições de tempo real. E a falha bizantina é a mais difícil de tratar: o processo continua respondendo, mas de forma arbitrária ou até maliciosa, podendo enviar informações contraditórias para diferentes partes do sistema. Quanto mais severo o modelo de falha assumido, mais cara e complexa é a solução necessária para tolerá-lo.

Replicação Ativa/Passiva e Arquiteturas de SO Distribuído

Entre as estratégias de replicação, a distinção mais cobrada é entre replicação ativa e replicação passiva. Na replicação ativa, todas as réplicas processam cada requisição de forma independente e em paralelo — o que exige que o processamento seja determinístico, para que todas as réplicas cheguem ao mesmo resultado, mas em troca oferece resposta mais rápida e maior disponibilidade, já que não há um único ponto de decisão. Na replicação passiva, existe uma réplica primária que processa as requisições e propaga o novo estado para as réplicas secundárias (backups); é conceitualmente mais simples de implementar e não exige determinismo, mas cria um ponto único de decisão que precisa de um mecanismo de failover caso a primária falhe.

Do lado dos sistemas operacionais distribuídos, vale reter a evolução das arquiteturas de núcleo — do núcleo monolítico tradicional até o microkernel, que move serviços do núcleo para o espaço de usuário em busca de maior modularidade e tolerância a falhas — e os dois problemas centrais que um SO distribuído precisa resolver na prática: como escalonar processos entre múltiplas máquinas de forma eficiente, e como coordenar o acesso a um

sistema de arquivos compartilhado entre nós diferentes, mantendo a ilusão de um sistema de arquivos único mesmo quando os dados estão fisicamente espalhados.

Exercício de fixação: identificando o modelo certo

Considere o seguinte cenário: um sistema de reservas de assentos para eventos roda em três réplicas, hospedadas em cidades diferentes. Durante uma falha de rede que isola uma das réplicas por 40 segundos, o sistema continua aceitando novas reservas normalmente nas duas réplicas que permanecem conectadas entre si, e reconcilia os dados assim que a conexão com a terceira réplica é restabelecida — aceitando, como efeito colateral, que por alguns instantes réplicas diferentes possam mostrar contagens de assentos disponíveis levemente diferentes.

A primeira pergunta é qual modelo de consistência está sendo priorizado nesse projeto. A resposta é consistência eventual: o sistema opta por continuar disponível mesmo durante a partição de rede, aceitando que os dados fiquem temporariamente divergentes entre réplicas até que a reconciliação os alinhe novamente — em termos do CAP Theorem, essa é uma escolha clara pelo lado da Disponibilidade (A) em detrimento da Consistência forte e imediata (C), já que a Tolerância a Partição (P) não é opcional.

A segunda pergunta é qual estratégia de replicação está implícita na descrição. A resposta é replicação ativa: mais de uma réplica aceita escritas simultaneamente e de forma independente, com reconciliação posterior — um comportamento incompatível com replicação passiva, em que apenas uma réplica primária processaria as escritas e as demais apenas replicariam o estado já decidido por ela. Reconhecer esse tipo de pista no enunciado — quem decide, e quando a decisão é tomada — é exatamente o tipo de raciocínio cobrado em avaliação.

Orientação do trabalho final: onde estamos

O trabalho final da disciplina é um sistema distribuído simples que evolui ao longo do semestre em três checkpoints, cada um somando uma nota parcial e recebendo feedback antes da etapa seguinte — o mesmo modelo de entregas incrementais usado em cursos de projeto de universidades como MIT e Stanford, em vez de um trabalho único entregue de uma vez só no fim.

O Checkpoint 1 — comunicação cliente-servidor via sockets ou RPC/RMI, entregue em 08/09 — e o Checkpoint 2 — evolução do sistema para múltiplas réplicas, com discussão do modelo de consistência adotado, entregue em 29/10 — já foram concluídos e já receberam feedback. O foco a partir de agora é inteiramente o Checkpoint 3: adicionar tratamento de falhas ao sistema — detecção de nó fora do ar, mecanismo de failover ou um esquema simples de consenso —, com entrega final em 15/12, que inclui código completo, relatório final com o histórico do projeto e as decisões de arquitetura tomadas, e uma apresentação de até 10 minutos por grupo na Aula 31.

Na rubrica de avaliação, o Checkpoint 1 vale 25% da nota do trabalho, o Checkpoint 2 vale 30%, o Checkpoint 3 vale 35% — o maior peso individual entre as três etapas — e os 10% restantes correspondem à participação, avaliada por autoavaliação do grupo combinada com observação em aula. Vale lembrar que o trabalho corresponde a 60% da média final da disciplina ($Média\ Final = 0,4 \times Nota\ da\ P1 + 0,6 \times Nota\ do\ Trabalho$), o que reforça a importância de reservar tempo real de desenvolvimento para essa reta final.

Cronograma até a apresentação final

Depois desta Aula 27 (17/11), os dias 19, 24 e 26/11 correspondem ao período de avaliação institucional da faculdade — não há aula expositiva nesses dias, e o tempo deve ser usado pelos grupos para avançar no

desenvolvimento do Checkpoint 3. A Aula 28, em 01/12, é um laboratório dedicado ao desenvolvimento do trabalho, com o professor circulando entre os grupos para apoiar decisões de implementação. A Aula 29, em 03/12, é uma apresentação parcial: cada grupo mostra o progresso alcançado até ali e alinha dúvidas antes da reta final. A Aula 30, em 10/12, faz uma revisão geral de todo o semestre. E a Aula 31, em 15/12, é a apresentação final dos trabalhos, quando o Checkpoint 3 completo é entregue e cada grupo apresenta o projeto em até 10 minutos.

Próxima aula

A Aula 28 (01/12) é um laboratório de desenvolvimento do trabalho final: não há conteúdo novo, a aula inteira é dedicada a colocar a mão na massa no Checkpoint 3, com acompanhamento do professor em sala para tirar dúvidas de implementação de detecção de falhas, failover ou consenso. A Aula 29 (03/12), na sequência, é o momento de apresentar o progresso parcial e alinhar qualquer dúvida remanescente antes da entrega e apresentação final na Aula 31 (15/12).

Referências

TANENBAUM, A. S.; VAN STEEN, M. Sistemas Distribuídos: Princípios e Paradigmas. 2ª ed. São Paulo: Pearson Prentice Hall, 2008. Base de todo o 2º bimestre: algoritmos distribuídos, tolerância a falhas e sistemas operacionais distribuídos.

COULOURIS, G.; DOLLIMORE, J.; KINDBERG, T. Sistemas Distribuídos: Conceitos e Projeto. 4ª ed. Porto Alegre: Bookman, 2007. Referência complementar para consenso, replicação e arquiteturas de sistemas operacionais distribuídos.